부록 · DB 관리 명령어 100선

MongoDB Docker 설정 + 주요명령어 100선

Docker 설정 · 영구유지 · 사용자생성 · 외부포트 · Docker 관리 · 주요 명령 100개

이 문서에서 고친 곳

- #86~88 rs.status·rs.conf·rs.initiate 에 「복제 세트로 띄운 경우만」을 붙였습니다. 이 문서의 compose(단일 서버)에서는 오류가 납니다.

1. Docker 핵심 설정표

항목	설정
이미지	mongo:latest
컨테이너	mongodb
포트	27017:27017
호스트 data	./data
컨테이너 data	/data/db
자동재시작	restart: unless-stopped

2. 폴더 및 docker-compose.yml

```bash
mkdir -p ~/mongodb/data
cd ~/mongodb
```

```yaml
services:
  mongodb:
    image: mongo:latest
    container_name: mongodb
    restart: unless-stopped
    ports:
      - "27017:27017"
    environment:
      MONGO_INITDB_ROOT_USERNAME: "root"
      MONGO_INITDB_ROOT_PASSWORD: "강력한ROOT비밀번호"
    volumes:
      - ./data:/data/db
```

3. 환경설정 항목 비교표

설정	기능	설명
image	DB 이미지	mongo:latest 사용
container_name	이름	관리용 이름 mongodb
restart	자동복구	unless-stopped로 서버/Docker 재기동 후 자동 시작

설정	기능	설명
ports	외부접속	호스트 27017 → 컨테이너 27017
environment	초기화	최초 빈 데이터 디렉터리에서 초기 계정/DB 설정
volumes	영구저장	./data → /data/db

4. 데이터 영구유지 표

항목	설명
컨테이너 삭제	docker compose down 후에도 bind mount의 호스트 data 폴더는 남음
서버 재부팅	restart: unless-stopped로 자동 기동
data 보호	호스트 data 폴더를 직접 삭제하지 않음
백업	논리백업 + 스토리지/폴더 백업 권장
업그레이드	버전 호환성과 백업을 확인한 뒤 진행

5. 사용자 생성 및 권한

#	용도	명령
1	root 접속	docker exec -it mongodb mongosh -u root -p --authenticationDatabase admin
2	DB 전환	use wizdata
3	사용자 생성	db.createUser({user:'devuser',pwd:'password',roles:[{role:'readWrite',db:'wizdata'}]})
4	사용자 확인	db.getUsers()
5	비밀번호 변경	db.changeUserPassword('devuser','new_password')
6	사용자 삭제	db.dropUser('devuser')

6. 외부 포트 및 접속

항목	값/명령
기본 포트	27017
Rocky 방화벽	sudo firewall-cmd --permanent --add-port=27017/tcp
반영	sudo firewall-cmd --reload
리스닝 확인	sudo ss -lntp | grep 27017
Windows 확인	Test-NetConnection <서버IP> -Port 27017
URI	mongodb://devuser:password@<서버IP>:27017/wizdata
mongosh	mongosh 'mongodb://root:password@<서버IP>:27017/?authSource=admin'

7. Docker 관리/컨테이너 명령 25개

#	용도	명령
1	실행 컨테이너	docker ps
2	전체 컨테이너	docker ps -a
3	이미지 목록	docker images
4	Compose 설정검증	docker compose config
5	이미지 다운로드	docker compose pull
6	Compose 실행	docker compose up -d
7	Compose 상태	docker compose ps
8	Compose 중지	docker compose stop
9	Compose 시작	docker compose start
10	Compose 재시작	docker compose restart
11	Compose 제거	docker compose down
12	컨테이너 시작	docker start mongodb
13	컨테이너 중지	docker stop mongodb
14	컨테이너 재시작	docker restart mongodb
15	로그	docker logs mongodb
16	실시간 로그	docker logs -f mongodb
17	최근 100줄	docker logs --tail 100 mongodb
18	자원 사용	docker stats mongodb
19	상세정보	docker inspect mongodb
20	포트 매핑	docker port mongodb
21	마운트	docker inspect -f '{{json .Mounts}}' mongodb
22	쉘	docker exec -it mongodb bash
23	네트워크	docker network ls
24	Docker 디스크	docker system df
25	프로세스	docker top mongodb

8. MongoDB 주요 명령어 100개

#	분류	용도	명령
1	기본	DB 목록	show dbs
2	기본	현재 DB	db
3	기본	DB 전환	use wizdata
4	기본	컬렉션 목록	show collections

#	분류	용도	명령
5	기본	서버 상태	db.serverStatus()
6	기본	버전	db.version()
7	기본	호스트 정보	db.hostInfo()
8	기본	연결 상태	db.runCommand({connectionStatus:1})
9	기본	DB 통계	db.stats()
10	기본	컬렉션 통계	db.member.stats()
11	사용자	현재 DB 사용자	db.getUsers()
12	사용자	사용자 생성	db.createUser({user:'devuser',pwd:'password',roles:[{role:'readWrite',db:'wizdata'}]})
13	사용자	관리 역할 사용자	db.createUser({user:'admin2',pwd:'password',roles:[{role:'dbAdmin',db:'wizdata'}]})
14	사용자	사용자 정보	db.getUser('devuser')
15	사용자	비밀번호 변경	db.changeUserPassword('devuser','new_password')
16	사용자	역할 추가	db.grantRolesToUser('devuser',[{role:'dbAdmin',db:'wizdata'}])
17	사용자	역할 회수	db.revokeRolesFromUser('devuser',[{role:'dbAdmin',db:'wizdata'}])
18	사용자	사용자 삭제	db.dropUser('devuser')
19	사용자	인증	db.auth('devuser','password')
20	사용자	로그아웃	db.logout()
21	컬렉션	생성	db.createCollection('member')
22	컬렉션	Capped 생성	db.createCollection('log',{capped:true,size:1048576})
23	컬렉션	목록	db.getCollectionNames()
24	컬렉션	정보	db.getCollectionInfos()
25	컬렉션	삭제	db.member.drop()
26	컬렉션	이름 변경	db.member.renameCollection('members')
27	삽입	한 건	db.members.insertOne({name:'HONG',age:57})
28	삽입	여러 건	db.members.insertMany([{name:'LEE'},{name:'PARK'}])
29	조회	전체	db.members.find()
30	조회	한 건	db.members.findOne({name:'HONG'})
31	조회	조건	db.members.find({age:{$gte:50}})
32	조회	AND	db.members.find({age:{$gte:50},name:'HONG'})
33	조회	OR	db.members.find({$or:[{name:'HONG'},{name:'LEE'}]})
34	조회	IN	db.members.find({name:{$in:['HONG','LEE']}})
35	조회	NIN	db.members.find({name:{$nin:['HONG','LEE']}})
36	조회	GT	db.members.find({age:{$gt:50}})
37	조회	GTE	db.members.find({age:{$gte:50}})

#	분류	용도	명령
38	조회	LT	db.members.find({age:{$lt:60}})
39	조회	LTE	db.members.find({age:{$lte:60}})
40	조회	NE	db.members.find({age:{$ne:57}})
41	조회	EXISTS	db.members.find({email:{$exists:true}})
42	조회	REGEX	db.members.find({name:/^K/})
43	조회	필드 선택	db.members.find({},{name:1,age:1,_id:0})
44	조회	정렬	db.members.find().sort({age:-1})
45	조회	LIMIT	db.members.find().limit(10)
46	조회	SKIP	db.members.find().skip(10).limit(10)
47	조회	COUNT	db.members.countDocuments({})
48	조회	DISTINCT	db.members.distinct('age')
49	수정	updateOne	db.members.updateOne({name:'HONG'},{$set:{age:58}})
50	수정	updateMany	db.members.updateMany({age:{$gte:50}},{$set:{grade:'A'}})
51	수정	INC	db.members.updateOne({name:'HONG'},{$inc:{age:1}})
52	수정	UNSET	db.members.updateOne({name:'HONG'},{$unset:{grade:''}})
53	수정	RENAME	db.members.updateOne({name:'HONG'},{$rename:{name:'username'}})
54	수정	PUSH	db.members.updateOne({username:'HONG'},{$push:{tags:'docker'}})
55	수정	ADD TO SET	db.members.updateOne({username:'HONG'},{$addToSet:{tags:'mongodb'}})
56	수정	PULL	db.members.updateOne({username:'HONG'},{$pull:{tags:'docker'}})
57	수정	UPSERT	db.members.updateOne({username:'CHOI'},{$set:{age:40}},{upsert:true})
58	수정	replaceOne	db.members.replaceOne({username:'HONG'},{username:'HONG',age:59})
59	삭제	deleteOne	db.members.deleteOne({username:'HONG'})
60	삭제	deleteMany	db.members.deleteMany({age:{$lt:20}})
61	삭제	전체 삭제	db.members.deleteMany({})
62	집계	MATCH	db.members.aggregate([{$match:{age:{$gte:50}}}])
63	집계	GROUP	db.members.aggregate([{$group:{_id:'$age',count:{$sum:1}}}])
64	집계	SUM	db.orders.aggregate([{$group:{_id:null,total:{$sum:'$amount'}}}])
65	집계	AVG	db.members.aggregate([{$group:{_id:null,avgAge:{$avg:'$age'}}}])
66	집계	MAX	db.members.aggregate([{$group:{_id:null,maxAge:{$max:'$age'}}}])
67	집계	MIN	db.members.aggregate([{$group:{_id:null,minAge:{$min:'$age'}}}])
68	집계	SORT	db.members.aggregate([{$sort:{age:-1}}])
69	집계	LIMIT	db.members.aggregate([{$limit:10}])
70	집계	PROJECT	db.members.aggregate([{$project:{name:1,age:1,_id:0}}])

#	분류	용도	명령
71	집계	LOOKUP	db.orders.aggregate([{$lookup: {from:'members',localField:'memberId',foreignField:'_id',as:'member'}}])
72	인덱스	인덱스 생성	db.members.createIndex({name:1})
73	인덱스	복합 인덱스	db.members.createIndex({name:1,age:-1})
74	인덱스	유니크	db.members.createIndex({email:1},{unique:true})
75	인덱스	텍스트	db.members.createIndex({bio:'text'})
76	인덱스	목록	db.members.getIndexes()
77	인덱스	삭제	db.members.dropIndex('name_1')
78	인덱스	전체 삭제	db.members.dropIndexes()
79	실행계획	explain	db.members.find({age:57}).explain('executionStats')
80	관리	현재 작업	db.currentOp()
81	관리	작업 종료	db.killOp(12345)
82	관리	Profiling 상태	db.getProfilingStatus()
83	관리	Profiling 설정	db.setProfilingLevel(1,{slowms:100})
84	관리	Ping	db.runCommand({ping:1})
85	관리	DB 삭제	db.dropDatabase()
86	관리	복제 상태	rs.status() ※ --replSet 으로 띄운 경우만
87	관리	복제 설정	rs.conf() ※ --replSet 으로 띄운 경우만
88	관리	복제 초기화	rs.initiate() ※ --replSet 으로 띄운 경우만
89	관리	로그 조회	db.adminCommand({getLog:'global'})
90	관리	FCV 조회	db.adminCommand({getParameter:1,featureCompatibilityVersion:1})
91	백업	mongodump	mongodump --uri='mongodb://root:password@localhost:27017/?authSource=admin' --out=backup
92	복원	mongorestore	mongorestore --uri='mongodb://root:password@localhost:27017/?authSource=admin' backup
93	Import	JSON 가져오기	mongoimport --db wizdata --collection members --file members.json --jsonArray
94	Export	JSON 내보내기	mongoexport --db wizdata --collection members --out members.json
95	쉘	스크립트 실행	mongosh 'mongodb://localhost:27017/wizdata' script.js
96	쉘	eval	mongosh --eval "db.adminCommand({ping:1})"
97	CRUD	findOneAndUpdate	db.members.findOneAndUpdate({name:'LEE'},{$set:{age:50}},{returnDocument:'after'})
98	CRUD	findOneAndDelete	db.members.findOneAndDelete({name:'LEE'})
99	CRUD	bulkWrite	db.members.bulkWrite([{insertOne:{document:{name:'A'}}},{updateOne:{filter:{name:'A'},update:{$set:{age:1}}}}])
100	검증	validate	db.members.validate({full:true})

9. 운영 주의사항

- MongoDB 공식 데이터 경로는 /data/db임.
- root 초기화 환경변수는 빈 data 최초 초기화 때 적용됨.
- 27017 외부 노출 시 인증과 IP 제한이 필수적임.
